tupLe conversion in usAinboltz for PoINting

Alexandre Vannereux

Theory elements for the creation of builders. The goal of this article is to give some theoric tools to study builder pointing.

Builders use nested tuples as arguments, the first question is then to understand at which condition a tuple can be considered valid for a rule. Then, it will be demonstrated that pointed tuple are converted by the algorithm to non pointed tuples. Finally, unpointing need to ensure that a non pointed tuple of size n (as a combinatorial object) has exactly n pointed tuple of size n mapped to it by the algorithm.

Some definitions elements:

- a python tuple is noted with a prefix Tp: (1, 2) becomes Tp(1, 2). Mathematical couples are noted normally.

Some aspects present in the real algorithms are simplified:

- there is a $Zero$ rule which corresponds to the empty class. This class will be used instead of simplifying as done in the real algorithms.
- we assume that builders always take as argument tuples and return tuples. For iterated rules, it is assumed that lists are tuples. For other objects (which are not tuples nor lists), they are considered to be a singleton with a single element.

We first define sub-right-rules:

Definition 1 *A sub-right-rule (which can be read as a sub rule of the right of a production rule in a grammar) is defined by induction:*

$$Atom(),\ Epsilon(),\ Marker(name),\ RuleName(name),\ Zero()\ \textit{for name a character chain.}$$

are sub-right-rules.

If R, R_1, ..., R_n are sub-right-rules,

$$Union(R_1, ..., R_n), Product(R_1, ..., R_n), Seq(R), Set(R), Cycle(R)$$

are sub-right-rules.

To help with understanding, here is the grammar associated with sub-right-rules:

$$
\begin{aligned}
R \rightarrow & Atom() \mid \\
& Epsilon() \mid \\
& Marker(name) \mid \\
& RuleName(name) \mid \\
& Zero() \mid \\
& Union(R, ..., R) \mid \\
& Product(R, ..., R) \mid \\
& Seq(R) \mid \\
& Set(R) \mid \\
& Cycle(R)
\end{aligned}
$$

For R a sub-right-rule, we say that S another sub-right-rule appear in R if R can be obtained by induction with S as one of R base element.

Example 1 *$R = Product(Atom(), Epsilon())$ is a sub-right-rule.*
$S_1 = Atom()$ appears in R because $Product(S_1, Epsilon()) = R$.
$S_2 = Zero()$ doesn't appear in R.

From this definition we can define a grammar.

Definition 2 *A grammar G is a set of production rules where a production rule is a couple (A, R) noted $A \rightarrow R$. A is a RuleName called a non terminal of G and R is a sub-right-rule called a right-rule of G.*
We say that a sub-right-rule S appears in G if it appears in a right-rule of G.

An additional condition on G to be considered a grammar is that all sub-right-rules A which are RuleNames also are non terminals of G and each non terminal G is only associated with a single right-rule.

We define the pointing algorithm on a sub-right-rule. Even though this code looks like python, there are pseudo-code elements. We could say this is pseudo-python.

```
#Some shortcuts are allowed: None−1 = None; (i−j) = max(0, i−j). Very useful for sizes constraints.
def Point(S: SubRightRule) −> SubRightRule:
    switch S:
        case Atom():
            return Atom()
        case Epsilon() | Marker(_) | Zero():
            return Zero()
        case Union(R1, ..., Rn):
            return Union(Point(R1), ..., Point(Rn))
        case Product(R1, ..., Rn):
            return Union(
                Product(Point(R1), R2, ..., Rn),
                ...,
                Product(R1, ..., Point(Rn))
        case RuleName(name):
            return RuleName(name + "_P")
        case Set(R, leq = i, geq = j):
            return Product(Point(R), Set(R, leq = i−1, geq = j−1))
        case Cycle(R, leq = i, geq = j):
            return Product(Point(R), Seq(R, leq = i−1, geq = j−1))
        case Seq(R, leq = None, geq = None):
```

```
        return Union(Product(Seq(R, eq = 0), Point(R), Seq(R)))
    case Seq(R, leq = i, geq = j):
        #with i != None.
        return Union(
            Product(Seq(R, eq = 0), Point(R), Seq(R, leq = i−1, geq = j−1)),
            Product(Seq(R, eq = 1), Point(R), Seq(R, leq = i−2, geq = j−2)),
            ...,
            Product(Seq(R, eq = i−1), Point(R), Seq(R, leq = 0, geq = j−i)))
    case Seq(R, leq = None, geq = j):
        #with j != None
        return Union(
            Product(Seq(R, eq = 0), Point(R), Seq(R, leq = None, geq = j−1)),
            ...,
            Product(Seq(R, eq = j−2), Point(R), Seq(R, leq = None, geq = 1)),
            Product(Seq(R, geq = j−1), Point(R), Seq(R)))
```

Definition 3 (Pointing) *For any sub-right-rule S, S pointed noted $S^\bullet$ is $S^\bullet = Point(S)$.*
For a grammar $G = \{A_1 \to R_1, ..., A_n \to R_n\}$, $G^\bullet = \{A_1 \to R_1, ..., A_n \to R_n, A_1^\bullet \to R_1^\bullet, ..., A_n^\bullet \to R_n^\bullet\}$.

Example 2 (Functional graph) *We define the grammar of functional graphs G:*

$$\begin{aligned} G = \{ & \\ & S \to && Set(Cycle(T)) \\ & T \to && Atom() \times Set(T) \\ \} & \end{aligned}$$

Then the pointed grammar $G^\bullet$ is:

$$\begin{aligned} G^\bullet = \{ & \\ & S \to && Set(Cycle(T)) \\ & T \to && Atom() \times Set(T) \\ & S^\bullet \to && T^\bullet \times Seq(T) \times Set(Cycle(T)) \\ & T^\bullet \to && Atom() \times Set(T) + Atom() \times T^\bullet \times Set(T) \\ \} & \end{aligned}$$

Now we define the tuple unpointing algorithm.

```
def UnpointTp(t: tuple, S: SubRightRule, stop_at_rulename: bool) −> tuple:
    switch S:
        case Union(R1, ..., Rn):
            Tp(i, tu) = t #This can raise an error if t has the wrong format.
            return Tp(i, UnpointTp(Ri, tu, stop_at_rulename))
        case Product(R1, ..., Rn):
            Tp(i, Tp(t1, ..., tn)) = t
            return Tp(t1, ..., UnpointTp(Ri, ti, stop_at_rulename), ..., tn)
        case RuleName(A):
            if stop_at_rulename:
                return t
            else:
                #if A is associated with the right rule R in the grammar,
                return UnpointTp(t, R, False)
        case Cycle(R) | Set(R):
            Tp(t1, Tp(t2, ..., tn))
```

```
        return Tp(Unpoint(R, t1, stop_at_rulename), t2, ..., tn)
    case Seq(R):
        Tp(_, Tp(Tp(t_left_1, ..., t_left_n), t_pointed, Tp(t_right_1, ..., t_right_m))) = t
        return Tp(t_left_1, ..., t_left_n, UnpointTp(R, t_mid, stop_at_rulename), t_right_1, ..., t_right_m)
    case Epsilon() | Marker(_) | Zero():
        raise Exception("Error")
```

Definition 4 (Acceptable tuple for a sub-right-rule.) *A context Γ is a couple (G, M) with G a grammar and M is a mapping from the set of non terminals in G to builders. For $A \to R$ a production rule in G, the builder $\mathcal{B} = M(A)$ is a function from the set of right-hand acceptable tuples for R to a set $E_{\mathcal{B}}$.*

For Γ a context, S a sub-right-rule of G, t^v and t^b two tuples, we say that (t^v, t^b) is acceptable for S and we note $\Gamma : S \vdash (t^v, t^b)$ if this proposition can be deduced from the following inference rules:

$$\frac{}{\Gamma : Epsilon() \vdash (Epsilon(), Epsilon())} \; Epsilon$$

$$\frac{}{\Gamma : Atom() \vdash (Atom(), Atom())} \; Atom$$

$$\frac{}{\Gamma : Marker('a') \vdash (Marker('a'), Marker('a'))} \; Marker$$

$$\frac{i \in [1; n] \quad \Gamma : S_i \vdash (t^v, t^b)}{\Gamma : S_1 + ... + S_n \vdash (Tp(i, t^v), Tp(i, t^b))} \; Union \; n \geq 2$$

$$\frac{\Gamma : S_1 \vdash (t_1^v, t_1^b) \quad ... \quad \Gamma : S_n \vdash (t_n^v, t_n^b)}{\Gamma : S_1 \times ... \times S_n \vdash (Tp(t_1^v, ..., t_n^v), Tp(t_1^b, ..., t_n^b))} \; Product \; n \geq 2$$

$$\frac{\Gamma : S \vdash (t_1^v, t_1^b) \quad ... \quad \Gamma : S \vdash (t_n^v, t_n^b)}{\Gamma : IterOP(S) \vdash (Tp(t_1^v, ..., t_n^v), Tp(t_1^b, ..., t_n^b))} \; IterOp \; IterOp \in \{Seq, Set, Cycle\}$$

$$\frac{A \to R \in G \quad \Gamma : R \vdash (t^v, t^b)}{\Gamma : A \vdash (t^v, M(A)(t^b))} \; Simplification$$

The b in t^b stands for builder because t^b is obtained by applying builders on the tuple. The v in t^v stands for vanilla because no builders have been used on t^v.

t^v is called a left-hand acceptable tuple for S and t^b a right-hand acceptable tuple. Sometimes, we will also say that t^v is a left-hand acceptable tuple associated with t^b for S and t^b is a a right-hand acceptable tuple associated with t^v for S.

We now define pointing.

Definition 5 (Context pointing) *For a context $\Gamma = (G, M)$, the pointed context $\Gamma^\bullet$ is $\Gamma^\bullet = (G^\bullet, M^\bullet)$. For $A \to R \in G$, $M^\bullet(A) = M(A)$ and $M^\bullet(A^\bullet)(t^{b\bullet}) = M(A)(UnpointTp(t^{b\bullet}, R, True))$ where $t^{b\bullet}$ is a right acceptable tuple for $R^\bullet$. As it will be seen later, $M(A)(UnpointTp(t^{b\bullet}, R, True))$ is correctly defined.*

The unpointing algorithm is different for left-hand and right-hand tuples. Indeed the left-hand t^v corresponds to our intuitive vision of a combinatorial structure represented by a tuple, $UnpointTp(t^v, A, False)$ convert the entire tuple into an unpoinited tuple. On the other hand, the right-hand t^b is closer to what is really implemented, the builders functions create entirely new objects. Those builders are called repeatedly and unpointing the entire tuple directly is not possible, instead we need to unpoint each time we call a builder function. That's why we stop unpointing at rulenames.

Example 3 (Binary trees) *We consider the following grammar for rooted binary trees:*

$$G = \{ \quad T \to \quad Atom() + Atom() \times T \times T \quad \}$$

We also need a builder for T, the height of the tree should be enough:

```
def height(tp):
    (i, content) #this is an union.
    if i == 0: #a leaf
        return 0
    (_, left_height, right_height) = content
    return max(left_height, right_height) + 1
```

We then have the context $\Gamma = \{G, \{T : height\}\}$.

We write $Z = Atom()$.

We can consider the tree $t^v = (0, ((0, Z), (0, Z)))$ of height $t^b = 1$. We want to prove that (t^v, t^b) is acceptable for T. We first demonstrate the leaf rule:

$$\frac{}{\Gamma : T \vdash ((0, Z), 0)}\ Leaf$$

which can be obtained by the following proof

$$\frac{\dfrac{\dfrac{}{\Gamma : Z \vdash (Z, Z)}\ Atom}{\Gamma : Z + Z \times T \times T \vdash ((0, Z), (0, Z))}\ Union}{\Gamma : T \vdash ((0, Z), 0)}\ Simp.$$

Now the main proof:

$$\frac{\dfrac{\dfrac{\dfrac{}{\Gamma : Z \vdash (Z, Z)}\ Atom \quad \dfrac{}{\Gamma : T \vdash ((0, Z), 0)}\ Leaf \quad \dfrac{}{\Gamma : T \vdash ((0, Z), 0)}\ Leaf}{\Gamma : Z \times T \times T \vdash (((0, Z), (0, Z)), (Z, 0, 0))}\ Product}{\Gamma : Z + Z \times T \times T \vdash (t^v, (1, (Z, 0, 0)))}\ Union}{\Gamma : T \vdash (t^v, 1)}\ Simp.$$

For integer which represent the element of an union, we write 0, 1, 2, etc... whereas for heights, we write 0, 1, 2, *etc...*

Finally we define the size of a left-hand tuple: $|t^v|_z$

$$\begin{aligned} |t^v|_z = & \\ & |t^v = Tp(i, t^{v'}) && \to |t^{v'}|_z \\ & |t^v = Tp(t^v_1, ..., t^v_n) && \to |t^v_1|_z + ... + |t^v_n|_z \\ & |t^v = Atom() && \to 1 \\ & |t^v = Epsilon() | t^v = Marker(\_) && \to 0 \end{aligned}$$

The theory to prove the three following theorems has now been established.

Theorem 1 (UnpointTp is an unpointing operation.) *Let $\Gamma = (G, M)$ be a context, R a sub-right-rule appearing in G and $t^{v\bullet}, t^{b\bullet}$ tuples.*

$\Gamma^\bullet : R^\bullet \vdash (t^{v\bullet}, t^{b\bullet})$ if and only if $UnpointTp(t^{v\bullet}, R, False)$ is correctly defined, $UnpointTp(t^{b\bullet}, R, True)$ is correctly defined and $\Gamma : R \vdash (UnpointTp(t^{v\bullet}, R, False), UnpointTp(t^{b\bullet}, R, True))$.

Corollary 1 *For a context $\Gamma = (G, M)$, $A \to R \in G$, $t^{b\bullet}$ a tuple. If $t^{b\bullet}$ is a right-hand acceptable tuple for $R^\bullet$. Then $M(A)(UnpointTp(t^{b\bullet}, R, True))$ is correctly defined. Therefore $M^\bullet(A^\bullet)$ is correctly defined.*

Theorem 2 (UnpointTp respects the size of tuples) *Let $\Gamma = (G, M)$ be a context, R a sub-right-rule appearing in G and $t^{v\bullet}, t^{b\bullet}$ tuples such that $\Gamma^\bullet : R^\bullet \vdash (t^{v\bullet}, t^{b\bullet})$. Then $UnpointTp(t^{v\bullet}, R, False)|_z = |t^{v\bullet}|_z$.*

Theorem 3 (For n tuples pointed of size n there is one non pointed tuple) *Let $\Gamma = (G, M)$ be a context, R a sub-right-rule appearing in G and t^v, t^b tuples such that $\Gamma : R \vdash (t^v, t^b)$. Then $|\{t^{v\bullet} | UnpointTp(t^{v\bullet}, R, False) = t^v$ and $t^{v\bullet}$ is left-hand acceptable for $R^\bullet\}| = |t^v|_z$.*

All three theorems will be proven by an induction on sub-right-rules.
But first we need to prove a small lemma.

Lemme 1 *Let $\Gamma = (G, M)$ a context, R a sub-right-rule appearing in G and (t^v, t^b). We have that:*
$\Gamma^\bullet : R \vdash (t^v, t^b)$ if and only if $\Gamma : R \vdash (t^v, t^b)$.

Proof 1 *The pointing operation create new productions rules but doesn't modify any production rule already present nor the mapping on non terminal present before the pointing. In other words: $G \subset G^\bullet$ and $M \subset M^\bullet$. Moreover the proof tree for $\Gamma^\bullet : R \vdash (t^v, t^b)$ can't involve any pointed element because no inference rule allow that. Therefore $\Gamma^\bullet : R \vdash (t^v, t^b)$ and $\Gamma : R \vdash (t^v, t^b)$ have the same valid proof tree.*

Proof 2 (UnpointTp is an unpointing operation.) *Base cases:*

- *$R = Epsilon()$ then $R^\bullet = Zero()$, there are no inference rules associated with $Zero()$, therefore there are no $(t^{v\bullet}, t^{b\bullet})$ acceptable for R. On the other hand, $UnpointTp(t^{v\bullet}, Epsilon(), False)$ raises an exception.*
- *$Marker()$ and $Zero()$ are very similar to $Epsilon()$.*
- *$R = Atom()$ therefore $R^\bullet = Atom()$. Then $UnpointTp(t^{v\bullet}, R, False) = t^{v\bullet}$ and $UnpointTp(t^{b\bullet}, R, True) = t^{b\bullet}$. If $\Gamma^\bullet : Atom() \vdash (t^{v\bullet}, t^{b\bullet})$, then we can apply the lemma to directly obtain the direct implication. The inverse implication is identical.*

Recursive cases, to avoid too much repetition, there is only a proof for unions, and rulenames:

- *$R = R_1 + ... + R_n$, then $R^\bullet = R_1^\bullet + ... + R_n^\bullet$. The only inference rule for union gives that $t^{v\bullet} = (i, t'^{v\bullet})$, $t^{b\bullet} = Tp(i, t'^{b\bullet})$ with $1 \leq i \leq n$ and $\Gamma^\bullet : R_i^\bullet \vdash (t'^{v\bullet}, t'^{b\bullet})$. R appears in G, therefore R_i appears in G. By hypothesis, $\Gamma : R_i \vdash (UnpointTp(t'^{v\bullet}, R_i, False), UnpointTp(t'^{b\bullet}, R_i, True))$ we conclude by applying the inference rule for union and simplifying the UnpointTp expression. The inverse implication is identical.*
- *If $R = A$ is a rulename such that $A \to S \in G$, the corresponding inference rule is the simplification. Therefore there exists a right hand acceptable tuple $t'^{b\bullet}$ associated with $t^{v\bullet}$ for $S^\bullet$ such that $M^\bullet(A^\bullet)(t'^{b\bullet}) = t^{b\bullet}$. We can apply the hypothesis of induction on S:*

$$\Gamma : S \vdash (Unpoint(t^{v\bullet}, S, False), Unpoint(t'^{b\bullet}, S, True))$$

$M(A)(UnpointTp(t'^{b\bullet}, R, True)) = M^\bullet(A^\bullet)(t'^{b\bullet}) = t^{b\bullet} = UnpointTp(t^{b\bullet}, A, True)$. By applying the inference rule for simplification,

$\Gamma : A \vdash (UnpointTp(t^{v\bullet}, A, False), M(A)(UnpointTp(t'^{b\bullet}, R, True))) = (UnpointTp(t^{v\bullet}, A, False), UnpointTp(t^{b\bullet}, A, Tr$

The inverse implication is identical.

Proof 3 (Unpoint respects the size of tuples) *Base cases*

- $R = Epsilon()$, *as explained above, this situation can't occur.*
- *The same argument can be used for* $Marker(\_)$ *and* $Zero()$
- $R = Atom()$, *therefore* $t^{v\bullet} = t^{b\bullet} = Atom()$, *and* $UnpointTp(., R,)$ *is the identity.*

Recursive cases (again on union and rulenames).

- $R = R_1 + ... + R_n$, *therefore* $t^{v\bullet} = Tp(i, t'^{v\bullet})$. *By induction,* $|UnpointTp(t'^{v\bullet}, R_i, False)|_z = |t'^{v}\bullet|_z$. *Knowing that* $|(i, t'^{v\bullet})|_z = |t'v\bullet|_z$ *and* $UnpointTp(Tp(i, t'^{v\bullet}), R, False) = UnpointTp(t'^{v\bullet}, R_i, False)$, *we have enough to conclude.*
- *If* $R = A$ *is a rulename such that* $A \to S \in G$, *the corresponding inference rule is the simplification. Moreover* $UnpointTp(, A, True)$ *is the identity which is enough to conclude by induction.*

Proof 4 (For n tuples pointed of size n there is one non pointed tuple) *Base cases:*

- *If* $R = Epsilon()$ *(or* $Marker()$ *or* $Zero()$*), then* $|t^v|_z = 0$ *and there are no* $t^{v\bullet}$ *right-hand acceptable for* $R^\bullet$, *therefore* $|\{t^{v\bullet} | UnpointTp(t^{v\bullet}, R, False) = t^v \text{ and } t^{v\bullet} \text{ is left-hand acceptable for } R^\bullet\}| = 0$.
- *If* $R = Atom()$, *then* $t^v = Atom()$ *and* $|t^v|_z = 1$. *There is exactly one right-hand acceptable tuple* $t^v\bullet$ *such that* $UnpointTp(t^{v\bullet}, R, False) = t^v$: $t^{v\bullet} = Atom()$. *This is enough to conclude.*

Recursive cases (only for product):

- *If* $R = R_1 \times ... \times R_n$, *then* $t^v = Tp(t_1^v, ..., t_n^v)$. *For any* $t^{v\bullet}$ *such that* $Unpoint(t^{v\bullet}, R^\bullet, False) = t^v$, *we know that* $t^v\bullet$ *is of the form:*

$$t^{v\bullet} = Tp(i, Tp(t_1^v, ..., t_{i-1}^v, t_i^{v\bullet}, t_{i+1}^v, ..., t_n^v))$$

Therefore,

$$\{t^{v\bullet} | UnpointTp(t^{v\bullet}, R, False) = t^v \text{ and } t^{v\bullet} \text{ is left-hand acceptable for } R^\bullet\} =$$
$$\uplus_{i=1}^n \{t_i^{v\bullet} | UnpointTp(t_i^{v\bullet}, R, False) = t_i^v \text{ and } t_i^{v\bullet} \text{ is left-hand acceptable for } R_i^\bullet\}$$

. We can conlude by applying the hypothesis of induction on each R_i.